

Data Structures Project Report

Library Management System

1. Introduction

This project presents a **Library Management System** developed using the **C++ programming language**.

The project simulates a real-world library environment through the effective use of various data structures, including a **singly linked list**, **doubly circular linked list**, **circular linked list**, **queue**, **stack**, **binary search tree**, and **hash table**.

The main objective of the application is to enable both library staff and users to efficiently manage book-related operations. The system integrates numerous features such as adding, deleting, updating, borrowing, returning, rating, and reporting books.

2. Data Structures Used and Their Purposes

- **Singly Linked List (BookManager):** Used for adding, deleting, updating, and listing books in the library.
- **Doubly Circular Linked List (UserCollection):** Used for creating and managing a user's personal book collection.
- **Circular Linked List (CircularList):** Used for storing borrowed, returned, and available books.
- **Stack (UserStack):** Used for tracking books borrowed and returned by users.
- **Queue (RequestQueue):** Used for processing book requests submitted by users and staff in sequential order.
- **Binary Search Tree (TreeNode):** Used for filtering and generating reports based on book attributes such as publication year, borrowing period, and publisher.
- **Hash Table (RatingHashTable):** Used for storing user ratings and displaying highly rated books.

3. General Features of the System

- User and Staff Login
- Adding, deleting, and updating books (title, author, publication year, etc.)
- Adding and viewing book comments
- Creating and managing personal collections
- Borrowing and returning books
- Adding and viewing book damage notes
- Book request system (academic / regular)
- Book rating and recommendation system
- Tree-based reporting according to publication year, borrowing duration, publisher, language, and page count

4. System Workflow

1. The user logs into the system.
2. The user borrows a book from the available books list.
3. The borrowed book is recorded in the appropriate stack structure.
4. When the user returns the book, it is transferred to the returned-books list and updated in the user's stack.
5. The user submits a rating, which is stored in the hash table.
6. Using the tree-based reporting system, the user can filter borrowed books and generate reports.

5. Conclusion

This project demonstrates how data structures can be applied effectively in real-world applications.

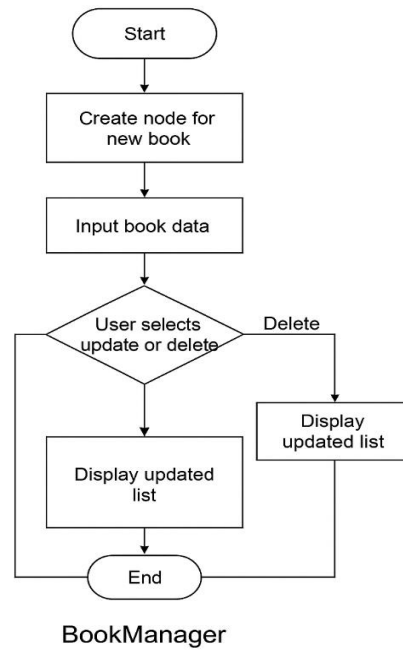
The developed system simulates both user and staff operations using different data structures while following an object-oriented programming approach.

All required modules were successfully implemented, and the system was tested to ensure stable and reliable operation.

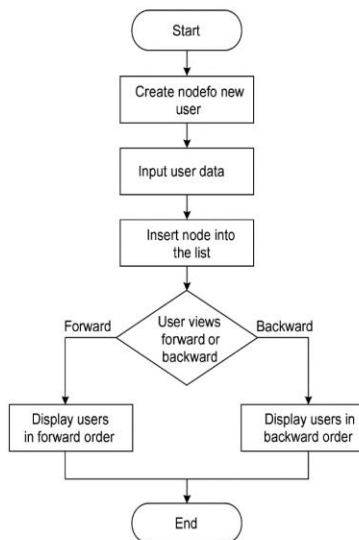
```

16 // =====
17 // BOOK MANAGER MODULE
18 // Handles operations related to book records in the library.
19 // Uses a singly linked list to store and manage books.
20 // =====
21 class BookManager {
22 public:
23     struct Book {
24         int id;
25         string title;
26         string author;
27         int pages;
28         int year;
29         string publisher; // ("TD", "EM")
30         string damageNote;
31         time_t addDate; // Date and time when the book was added
32         Book* next; // Pointer to the next book in the linked list
33     };
34     // Constructor to initialize book attributes
35     Book(int i, const string& t, const string& a, int p, int y, const string& pub)
36         : id(i), title(t), author(a), pages(p), year(y), publisher(pub),
37           damageNote(""), addDate(time(nullptr)), next(nullptr) {}
38 };
39 private:
40     Book* head; // Pointer to the first book in the list
41     int count; // Total number of books
42     // Constructor to initialize an empty list
43     BookManager() : head(nullptr), count(0) {}
44     void addBook(int id, const string& title, const string& author, int pages, int year) {
45         if (findBook(id) != nullptr) {
46             cout << "Error: Book ID already exists!\n";
47             return;
48         }
49         // Auto assign publisher based on even/odd ID
50         string publisher = (id % 2 == 0) ? "TD" : "EM";
51         // Create and insert the new book at the beginning of the list
52         Book* newBook = new Book(id, title, author, pages, year, publisher);
53         newBook->next = head;
54         head = newBook;
55         count++;
56     }
57     // Find a book by ID
58     Book* findBook(int id) const {
59         Book* current = head;
60         while (current != nullptr) {
61             if (current->id == id) return current;
62             current = current->next;
63         }
64         return nullptr;
65     }
66     // Display all books sorted by ID in ascending order
67     void listBooks() const {
68         vector<Book*> books;
69         Book* current = head;
70         while (current != nullptr) {
71             books.push_back(current);
72             current = current->next;
73         }
74     }
75 };

```



Singly Linked List



Doubly Circular Linked List
(UserCollection)

```

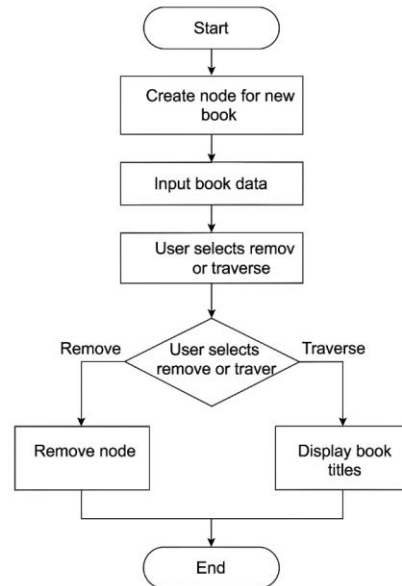
349 };
350 // ===== User Book Collection =====
351 // This module represents the user's personal book collection.
352 // It is implemented as a circular doubly linked list.
353 class UserCollection {
354 private:
355     struct UserBook {
356         int id;
357         string title;
358         string author;
359         int pages;
360         int year;
361         bool isHead; // Head status (true = read)
362         UserBook* next;
363         UserBook* prev;
364     };
365     UserBook(int i, const string& t, const string& a, int p, int y)
366         : id(i), title(t), author(a), pages(p), year(y),
367           isHead(false), next(nullptr), prev(nullptr) {}
368 };
369 UserBook* head; // Head of the list
370 bool collectionCreated; // Whether the collection is initialized
371 public:
372     UserCollection() : head(nullptr), collectionCreated(false) {}
373     // Creates the user's collection if not already created
374     void createCollection() {
375         if (collectionCreated) {
376             cout << "You already have a collection.\n";
377             return;
378         }
379         collectionCreated = true;
380         cout << "Your personal book collection has been created!\n";
381     }
382     // Adds a book to the user's collection (at the end of the circular list)
383     void addBook(int id, const string& title, const string& author, int pages, int year) {
384         if (!collectionCreated) {
385             cout << "Please create a collection first.\n";
386             return;
387         }
388         UserBook* newBook = new UserBook(id, title, author, pages, year);
389         if (head == nullptr) {
390             head = newBook;
391             head->next = head;
392             head->prev = head;
393         }
394         else {
395             newBook->next = head;
396             newBook->prev = head->prev;
397             head->prev->next = newBook;
398             head->prev = newBook;
399         }
400         cout << "Book added to your collection!\n";
401     }
402 };

```

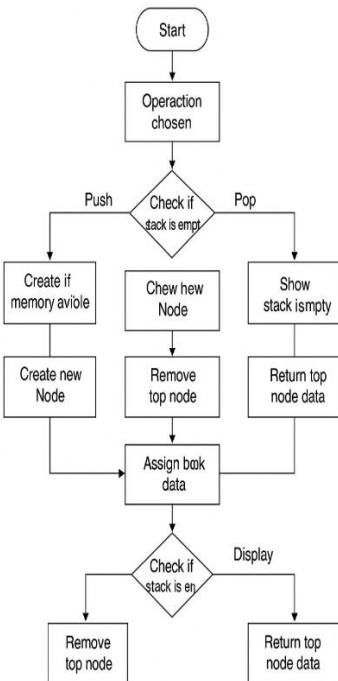
```

479 // ===== CircularList: Single Circular Linked List =====
480 // This class is used to manage circular lists such as borrowed, returned, or available books.
481 // =====
482 class CircularList {
483 private:
484     // Node represents a single book in the circular list.
485     struct Node {
486     int id;
487     string title;
488     string author;
489     int pages;
490     int year;
491     string borrowDate;
492     string returnDate;
493     Node* next;
494     // Constructor initializes node attributes
495     Node(int i, const string& t, const string& a, int p, int y)
496     : id(i), title(t), author(a), pages(p), year(y),
497     borrowDate("N/A"), returnDate("N/A"), next(nullptr) {}
498     };
499     Node* head; // Head pointer of the circular list
500 public:
501     void removeBook(int id);
502     // Returns the borrow date for a given book ID
503     string getBorrowDate(int id) const {
504     if (!head) return "N/A";
505     Node* current = head;
506     do {
507     if (current->id == id)
508     return current->borrowDate;
509     current = current->next;
510     } while (current != head);
511     return "N/A";
512     }
513     CircularList() : head(nullptr) {}
514     // Adds a borrowed book to the list and records the current date/time
515     void addBorrowedBook(int id, const string& title, const string& author, int pages, int year) {
516     Node* newNode = new Node(id, title, author, pages, year);
517     if (!head) {
518     head = newNode;
519     head->next = head;
520     }
521     else {
522     Node* temp = head;
523     while (temp->next != head) temp = temp->next;
524     temp->next = newNode;
525     newNode->next = head;
526     }
527     }
528     };
529

```



CircularList

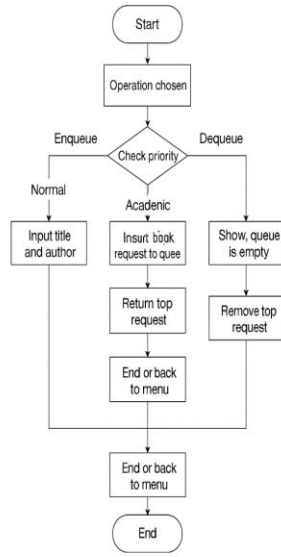


Yığın (UserStack)

```

625 // ===== Stack for User Borrowed/Returned Books =====
626 class UserStack {
627 public:
628     // Structure representing a node in the stack
629     struct StackNode {
630     int id;
631     string title;
632     string author;
633     StackNode* next;
634     // Constructor to initialize a book node
635     StackNode(int i, const string& t, const string& a)
636     : id(i), title(t), author(a), next(nullptr) {}
637     };
638     StackNode* top;
639 public:
640     // Constructor initializes the top to null
641     UserStack() : top(nullptr) {}
642     // Add a book to the top of the stack
643     void push(int id, const string& title, const string& author) {
644     StackNode* newNode = new StackNode(id, title, author);
645     newNode->next = top;
646     top = newNode;
647     }
648     // Remove the top book from the stack
649     void pop() {
650     if (top == nullptr) {
651     cout << "Stack is empty.\n";
652     return;
653     }
654     StackNode* temp = top;
655     top = top->next;
656     delete temp;
657     }
658     // Return the top book without removing it
659     StackNode* peek() const {
660     return top;
661     }
662     // Check if the stack is empty
663     bool isEmpty() const {
664     return top == nullptr;
665     }
666     // Display all books in the stack
667     void display() const {
668     if (top == nullptr) {
669     cout << "Stack is empty.\n";
670     return;
671     }
672     StackNode* temp = top;
673     cout << "\n--- Stack Books ---\n";
674     while (temp) {
675     cout << temp->id << " | " << temp->title
676     << " by " << temp->author << endl;
677     temp = temp->next;
678     }
679     }
680     };
681

```

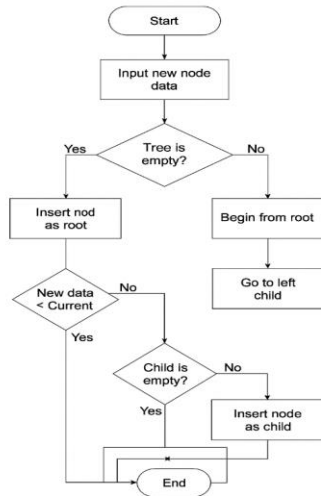


(Kuyruk (Queue)
(RequestQueue)

```

22 //=====
23 // MODULE 3: RequestQueue
24 // Purpose: Manages book requests (normal and academic)
25 //=====
26 class RequestQueue {
27 private:
28 // Structure to store each book request
29 struct BookRequest {
30 string title;
31 string author;
32 bool isAcademic;
33 int priority; // 1 = academic, 0 = normal
34 BookRequest* next;
35
36 BookRequest(string t, string a, bool academic)
37 : title(t), author(a), isAcademic(academic), priority(academic ? 1 : 0), next(nullptr) {}
38 };
39
40 BookRequest* front; // Points to the front of the queue
41 BookRequest* rear; // Points to the rear of the queue
42
43 public:
44 // Constructor: Initializes an empty queue
45 RequestQueue() : front(nullptr), rear(nullptr) {}
46
47 // Adds a new book request to the queue based on priority
48 void enqueue(string title, string author, bool isAcademic) {
49 BookRequest* newRequest = new BookRequest(title, author, isAcademic);
50 // Insert with higher priority (academic) at the front
51 if (!front || newRequest->priority > front->priority) {
52 newRequest->next = front;
53 front = newRequest;
54 if (rear == newRequest;
55 }
56 }
57 // Traverse to insert based on priority order
58 BookRequest* current = front;
59 while (current->next && current->next->priority >= newRequest->priority) {
60 current = current->next;
61 }
62 newRequest->next = current->next;
63 current->next = newRequest;
64 if (!newRequest->next) rear = newRequest;
65
66 cout << "Request added to queue!\n";
67 }
68
69 // Prints all current requests
70 void printRequests() const {
71 if (!front) {
72 cout << "No requests in the queue!\n";
73 return;
74 }
75 }

```



İkili Arama Ağacı (TreeNode)

```

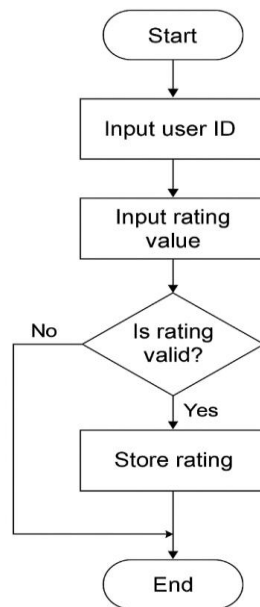
685 // ===== Global Tree Node Structure =====
686 // Structure used in the Binary Search Tree for reporting
687 struct TreeNode {
688 int id;
689 string title;
690 string author;
691 int pages;
692 int year;
693 string publisher;
694 string language;
695 time_t borrowDate;
696
697 TreeNode* left;
698 TreeNode* right;
699
700 // Constructor to initialize a tree node
701 TreeNode(int i, string t, string a, int p, int y, string pub, string lang, time_t date)
702 : id(i), title(t), author(a), pages(p), year(y),
703 publisher(pub), language(lang), borrowDate(date),
704 left(nullptr), right(nullptr) {}
705 };

```

```

1179 // Tree-based report filters for borrowed books
1180 void reportTreeMenu() {
1181 while (true) {
1182 cout << "\n----- TREE REPORT MENU ----- \n";
1183 << "1. Transfer Borrowed Books to Tree\n";
1184 << "2. Report: Borrow Duration < or >= 30 days\n";
1185 << "3. Report: Published Before or After 1950\n";
1186 << "4. [count char [43]] 3. Report: Published Before or After 1950\n";
1187 << "5. Search Online
1188
1189 int choice;
1190 cin >> choice;
1191 cin.ignore();
1192
1193 switch (choice) {
1194 case 1:
1195 addBorrowedBooksToTree();
1196 break;
1197 case 2:
1198 cout << "\n--- Borrow Duration Report ---\n";
1199 reportBooksByBorrowDuration(borrowTreeRoot);
1200 break;
1201 case 3:
1202 cout << "\n--- Report: Published Before or After 1950 ---\n";
1203 reportBooksByPublicationYear(borrowTreeRoot);
1204 break;
1205 case 4:
1206 cout << "\n--- Combined Report: Before 1950 AND Borrowed > 30 Days ---\n";
1207 reportBooksByYearAndDuration(borrowTreeRoot);
1208 break;
1209 case 5:
1210 return;
1211 default:
1212 cout << "Invalid choice.\n";
1213 }
1214 }
1215
1216 // ===== User Tree Filter Menu =====
1217 void reportUserTreeMenu() {
1218 // Transfer user's borrowed books from the stack into a binary tree
1219 transferUserBorrowedToTree();
1220
1221 while (true) {
1222 // Display tree filtering options to the user
1223 cout << "\n----- USER TREE FILTER MENU ----- \n";
1224 }
1225 }

```



Hash Tablo (RatingHashTable)

```

706 // ===== Hash Table for Book Ratings =====
707 class RatingHashTable {
708 private:
709     // Structure for a single rating entry in the hash table
710     struct RatingEntry {
711         int bookID;
712         string title;
713         int rating; // 1 to 5
714         RatingEntry* next;
715         // Constructor to initialize a rating entry
716         RatingEntry(int id, const string& t, int r)
717             : bookID(id), title(t), rating(r), next(nullptr) {}
718     };
719
720     static const int SIZE = 10; // Size of the hash table
721     RatingEntry* table[SIZE]; // Array of pointers to linked list buckets
722     // Hash function based on book ID
723     int hashFunction(int id) const {
724         return id % SIZE;
725     }
726
727 public:
728     // Constructor initializes all buckets to null
729     RatingHashTable() {
730         for (int i = 0; i < SIZE; ++i)
731             table[i] = nullptr;
732     }
733     // Insert a new rating into the hash table
734     void insert(int id, const string& title, int rating) {
735         if (rating < 1 || rating > 5) {
736             cout << " Invalid rating. Must be between 1 and 5.\n";
737             return;
738         }
739     }

```